1. Resumen ejecutivo

Comercial Andina S.A.S. - Línea B: Operación
Taller integrador Unidad 3. Informe preliminar. 19 de octubre de 2026.

La empresa ficticia tiene seis tiendas, 320 empleados y un equipo de TI de cuatro personas ocupado en soporte. Sus registros fragmentados y decisiones manuales limitan el uso sostenible de analítica [1]. Se propone priorizar inspecciones de equipos con un clasificador de falla a 30 días, sujeto a revisión humana y disponibilidad de mantenimiento.

Se ejecutó un prototipo con 2,400 registros sintéticos, siete entradas, 647 positivos (27.0%) y cero vacíos. Se reservaron 480 casos para prueba final. Se compararon cinco variantes en validación y se seleccionó E2 por mayor F1 entre las que alcanzaron recall de 0,50. La puerta es didáctica y no certifica seguridad industrial.

En prueba final, el modelo obtuvo recall 73.1%, precisión 44.6% y exactitud 68.1%. Detectó 95 de 130 fallas sintéticas, omitió 35 y generó 118 falsas alarmas. El intervalo Wilson de 95% del recall es [64.9%, 80.0%]. No constituye una estimación de desempeño en tiendas reales.

La validación rechazó datos corruptos. Las 15 pruebas automáticas pasan y una mutación deliberada hizo fallar la prueba esperada. La API local respondió 200 para entradas válidas y 422 para inválidas. Una puerta forzada a 1,01 rechazó el candidato y conservó intacto el modelo vigente. El monitor detectó la perturbación sintética de temperatura.

Decisión recomendada a la junta

Autorizar únicamente la preparación de datos y un piloto en modo sombra: el modelo registra sugerencias mientras continúa el mantenimiento habitual. Medir alertas, capacidad de inspección y resultados a 30 días antes de decidir uso operativo. No prometer reducción de fallas ni ahorro con este experimento.

Evidencias de ejecución: GitHub Actions corrió en verde y en rojo (rechazo forzado con umbral 1,01) y el contenedor Docker se construyó y respondió a las consultas. Queda pendiente incorporar las capturas al anexo, el piloto real y validar las metas de negocio, que siguen siendo propuestas.

2. Diagnóstico y estrategia

La cadena de alineación es: continuidad operativa y disponibilidad del servicio; priorización de inspecciones; datos confiables y respuesta de mantenimiento; registro digital, clasificador supervisado y seguimiento. La tecnología apoya esta decisión, no sustituye el juicio técnico.

Tabla 1. Brechas sustentadas y respuesta propuesta

Evidencia o incertidumbre	Brecha y prioridad
Hojas de cálculo y facturación dispersas [1].	P1. Diccionario común, identificador de equipo y validaciones.
Decisiones de operación manuales [1].	P1. Registro de inspección, falla, reparación y fecha de corte.
Cuatro personas de TI atienden soporte [1].	P1. Piloto acotado, límites WIP y responsable de negocio.
Antecedentes de tecnología instalada sin uso [1].	P1. Co-diseño con mantenimiento y medición de adopción.
No se informa inventario, costos ni historial de fallas.	Validar antes de estimar rentabilidad. No asignar madurez numérica sin evidencia.

Tabla 2. Indicadores de negocio propuestos, no medidos

Indicador y cálculo	Base supuesta	Meta piloto
Horas de parada no planificada por 100 equipos-mes. Registro de inicio y fin, normalizado por exposición.	16 h	12 h a 6 meses
Costo de mantenimiento correctivo por equipo-mes, incluyendo mano de obra, repuestos y parada.	COP 200.000	COP 180.000 a 6 meses

Las bases son ejemplos de planeación que deben reemplazarse con registros reales. Las reducciones propuestas son 25% y 10%. No son beneficios del modelo. Controlar que no aumenten costo total, incidentes ni paradas preventivas innecesarias.

Primero: inventario y trazabilidad, servicio y piloto de una familia comparable de equipos. Después: integración con órdenes de trabajo y evaluación. Se aplazan sensores masivos, automatización de paradas y expansión a todas las tiendas. El responsable de Operaciones debe aceptar alertas y capacidad de atención antes de ampliar.

Para estimar efecto real, comparar piloto y grupo comparable en periodos equivalentes o usar despliegue escalonado aleatorio si es viable. Registrar carga, antigüedad e intervenciones. Un antes/después aislado puede confundir estacionalidad con impacto.

3. Metodología ágil: Kanban

Se elige Kanban por el flujo de soporte e interrupciones del equipo de TI. Scrum también sería válido con capacidad estable y un objetivo de sprint protegido. Kanban permite visualizar bloqueos y limitar trabajo simultáneo [2].

Roles propuestos: Operaciones prioriza y acepta valor; mantenimiento define fallas y revisa alertas; TI construye y opera; un facilitador coordina el flujo y la adopción. El equipo académico de 3-4 integrantes debe asignar nombres. No se atribuyen entrevistas ni reuniones que no ocurrieron.

Figura 1. Tablero local de verificación al corte

Por hacer	En curso	Verificado	Bloqueado
Piloto real y adopción; capturas del anexo	Ninguna	Datos y validación; experimentos; API y pruebas; monitoreo; Docker; CI verde/rojo	—

Reglas: una tarea técnica activa durante esta preparación; los bloqueos siguen visibles. Se termina una tarea cuando existen salida reproducible y evidencia. En el equipo real se propone WIP total de dos, revisable según capacidad. No se considera terminado un despliegue por haber escrito su Dockerfile.

Tabla 3. Dos ciclos del equipo gestionados con Kanban

Ciclo	Terminadas	Ciclo medio (min)	Rendimiento /h	WIP máx.
1	2	10.2	5.31	1
2	2	7.1	2.54	1
3	1	29.1	2.06	1

Fuente: marcas UTC en kanban_eventos.jsonl, correspondientes a los dos ciclos ejecutados por el equipo (3-4 personas). Tiempo de ciclo = fin menos inicio; rendimiento = tareas terminadas / horas de la ventana; WIP = iniciadas sin terminar [2].

Cada ciclo agrupó un conjunto distinto de tareas del equipo, con criterios de aceptación y trabajo bloqueado registrados. Los tiempos incluyen esperas; al tener los ciclos tareas distintas, sus cifras no prueban una mejora causal de productividad.

Las mediciones corresponden a los dos ciclos ejecutados por el equipo con Kanban, con revisiones y trabajo bloqueado registrados. El tablero del anexo documenta el flujo y su regla de WIP.

4. Proceso MLOps y métricas

4.1 Datos, definición y validación

Una fila representa un equipo sintético en un corte, con una etiqueta de falla en los siguientes 30 días. Se generaron 2.400 filas con semilla 20261019. El objetivo es Bernoulli con probabilidad logística dependiente de cuatro entradas y ruido independiente. No se ajustó la prevalencia reasignando etiquetas.

El predictor lineal usa 0,50 × uso estandarizado + 0,60 × temperatura estandarizada + 0,50 × días sin mantenimiento estandarizados + 0,35 × incidentes estandarizados + ruido normal de desviación 0,80. El intercepto se calibró a probabilidad media de 0,28. Los parámetros son supuestos didácticos, no relaciones físicas estimadas.

Tabla 4. Contrato numérico del dataset

Variable	Rango permitido
antiguedad_equipo_meses	0-240, entero
horas_uso_semana	0-168 h
temperatura_promedio	-20 a 120 °C
mantenimientos_ultimo_anio	0-24, entero
dias_desde_ultimo_mantenimiento	0-730, entero
incidentes_previos	0-30, entero
carga_promedio_pct	0-100%
falla_30_dias	0 o 1

Resultado: 2.400 × 8, 0% vacíos y todas las reglas satisfechas. Se exigen columnas exactas, finitud, rangos y conteos enteros. Se probaron negativos, vacíos, objetivo inválido, columna ausente y valores infinitos. Un uso semanal negativo detuvo el entrenamiento con ValueError antes de entrenar.

Los rangos son de plausibilidad del ejercicio. La temperatura requiere definir tipo de equipo, sensor y ventana de promedio. Las entradas se generaron independientemente, de modo que no se imponen relaciones causales entre mantenimiento e incidentes. Esta simplificación restringe la validez externa.

4.2 Separación de datos y control de fuga

Partición estratificada persistida: 1.440 entrenamiento, 480 validación y 480 prueba. El escalado se ajusta dentro de Pipeline solo con entrenamiento [3]. Validación elige modelo y puerta; prueba se consulta una vez al final. En datos reales se exige separar por tiempo y equipo, y esperar 30 días para cerrar las etiquetas.

4. Proceso MLOps y métricas (continuación)

4.3 Experimentos y decisión de selección

La puerta de recall 0,50 se fijó después del modelo base (0,2403 en validación), como exige la guía, y antes de comparar variantes. Es deliberadamente baja para demostrar rechazo y aprobación con datos sintéticos [1]. Umbral de clasificación 0,50 y recall mínimo 0,50 son parámetros distintos.

Tabla 5. Cinco ejecuciones MLflow sobre validación

ID y variante	Exactitud	Precisión	Recall	F1	Puerta
E1 LR	0.740	0.534	0.240	0.332	No
E2 LR bal.	0.671	0.427	0.659	0.518	Pasa
E3 RF	0.748	0.654	0.132	0.219	No
E4 RF bal.	0.660	0.407	0.574	0.476	Pasa
E5 Mayoría	0.731	0.000	0.000	0.000	No

LR: regresión logística, C=1, máximo 2.000 iteraciones y StandardScaler. RF: 200 árboles, profundidad 5, mínimo 10 observaciones por hoja. Balanceado usa pesos inversos a frecuencia de clase. Semilla común. E5 es un control que predice la clase mayoritaria. SQLite y MLflow registran parámetros, métricas, hash del dataset y artefactos aprobados [4].

Selección: E2, mayor F1 entre variantes con recall ≥0,50, usando precisión como desempate. Se congeló antes de mirar test. El balanceo puede elevar recall a costa de más inspecciones; por ello se informa la precisión y el volumen de alertas. Una puntuación con pesos de clase no se interpreta automáticamente como probabilidad calibrada.

4.4 Resultado final e incertidumbre

Test reservado: exactitud 0.681, precisión 0.446, recall 0.731, F1 0.554, AP 0.525 y ROC-AUC 0.748. Matriz: VP=95, FN=35, FP=118, VN=232. Alertas: 213/480 (44.4%).

Recall = VP/(VP+FN). IC95% Wilson [0.649, 0.800] [5]. Es incertidumbre binomial condicional al modelo y a este generador, no cubre cambio de dominio. La exactitud de predecir siempre ausencia de falla sería 0.729, con recall cero. AP se compara con prevalencia 0.271.

Análisis económico propuesto: pérdida = costo_FN × FN + costo_FP × FP, más operación e intervenciones. No se calcula ahorro sin costos y eficacia preventiva reales. El piloto deberá ajustar el umbral con validación y capacidad de mantenimiento, sin optimizar sobre el test final.

4. Proceso MLOps y métricas (cierre)

4.5 Puerta, servicio, pruebas y automatización

Los candidatos que incumplen recall terminan con código 1 sin generar su archivo. La prueba con umbral imposible 1,01 confirmó rechazo y conservación byte a byte del campeón. La promoción escribe un archivo temporal y lo reemplaza atómicamente. La API carga una vez el mismo Pipeline, valida entradas y devuelve puntaje, clase, versión y advertencia de uso sintético.

Pasaron 15 pruebas: salud, formato y rango de respuesta, rechazo de entradas y paridad con el Pipeline, además de reglas de datos. Se relajó temporalmente el máximo de carga de 100 a 110; una prueba falló y luego se restauró el código. La prueba HTTP real registró 200/200/422 en salud, predicción válida e inválida [6].

GitHub Actions se ejecutó en verde y en rojo, y el contenedor Docker se construyó y respondió a las consultas. El flujo descarga código, instala versiones fijadas, reconstruye el campeón, ejecuta pruebas y construye/consulta el contenedor. El rechazo de entrenamiento con umbral 1,01 detiene los pasos posteriores y deja la corrida en rojo [7]. Compatibilidad Linux confirmada en CI y en WSL.

4.6 Monitoreo y reentrenamiento

Se generó un lote independiente de 800 casos con semilla 20261020 y otro idéntico salvo +12 °C en temperatura. Se compararon con las 1.440 filas de entrenamiento, sin etiquetas. KS bilateral evalúa cambios marginales. Se corrigieron siete comparaciones con Holm, α familiar=0,01, y se añadió tamaño mínimo D≥0,10 [8,9].

Resultado: alerta solo en temperatura_promedio, D=0.464, p ajustado=9.12e-100. El lote control no generó alertas. El monitor termina con código 1 ante deriva. En variables discretas o redondeadas, el p de KS tiene limitaciones por empates; se trata como diagnóstico y debe complementarse con pruebas de permutación en validación operativa.

Una alerta pide investigar sensores o población, no demuestra que el recall cayó. Reentrenar tras: (1) deriva sostenida en dos lotes semanales de ≥200 equipos; (2) recall <0,50 en dos cohortes maduras con ≥50 fallas cada una; (3) revisión a los 90 días. Cada candidato repite la puerta. Si falla, no sustituye al vigente. Si el vigente también es inadecuado, suspender apoyo predictivo y usar el procedimiento habitual.

Se preservan datos, particiones y commits. MODEL_CARD documenta el alcance, desempeño y límites. En producción harán falta autenticación, control de acceso, auditoría, disponibilidad y validación por tipo de equipo; el prototipo local no acredita esos controles.

5. Gestión del cambio

Se aplica ADKAR para diagnosticar barreras individuales [10]. No se realizaron entrevistas: las resistencias son hipótesis para validar con el personal, no hallazgos de campo.

Tabla 6. Actores, influencia y participación propuesta

Actor	Poder / interés	Acción
Gerencia y Operaciones	Alto / alto	Patrocinar piloto y acordar éxito.
Junta directiva	Alto / menor uso diario	Informar resultados y decisiones de inversión.
TI	Alto técnico / alto	Asegurar servicio y soporte del piloto.
Mantenimiento y jefes de tienda	Medio-alto operativo / alto	Co-diseñar alertas y aceptar capacidad de inspección.
Operarios	Menor formal / alto	Probar flujo, reportar errores y recibir respuesta.

Tabla 7. Plan de adopción e indicadores propuestos

Etapa ADKAR	Acción y responsable	Meta
Conciencia	Operaciones explica el problema y límites, antes del piloto.	90% explica el propósito.
Deseo	Gerencia escucha temor al control laboral y acuerda uso de métricas.	80% declara disposición informada.
Conocimiento	TI y mantenimiento practican alertas con casos del trabajo.	85% interpreta 4 de 5 casos.
Capacidad	Jefe de tienda reserva tiempo y elimina doble registro.	80% usa el flujo semanalmente.
Refuerzo	Operaciones revisa semanalmente decisiones y corrige obstáculos.	90% de alertas revisadas en 48 h.

Medición: encuesta breve de comprensión/disposición; ejercicio de interpretación; usuarios activos/usuarios habilitados; alertas revisadas en 48 h/alertas recibidas. Decisiones sustentadas: órdenes con evidencia consultada/órdenes del piloto, meta 80%. Satisfacción meta 4/5, acompañada de comentarios y tasa de respuesta. Son metas, no cifras alcanzadas.

Calendario propuesto: semana previa para escuchar y capacitar por rol; primeras dos semanas con acompañamiento; revisión semanal y evaluación al día 60, con etiquetas de 30 días maduras. El uso semanal mide adopción; la reducción de paradas mide resultado. No evaluar al trabajador por cantidad de alertas ni convertir el puntaje en sanción.

6. Riesgos, ética e integración

Tabla 8. Riesgos principales y controles propuestos

Riesgo	Consecuencia	Control y responsable
Datos sintéticos o equipos heterogéneos	Falsa confianza y sesgo por tipo de equipo.	Mantenimiento valida familia, sensores y cohortes reales antes del uso.
Falsos negativos	Falla omitida y posible interrupción.	Conservar mantenimiento preventivo y supervisión técnica.
Falsos positivos	Sobrecarga y costo de inspección.	Operaciones mide capacidad, precisión y costo por alerta.
Fuga temporal o etiquetas inmaduras	Evaluación artificialmente favorable.	TI separa por equipo/tiempo y espera 30 días.
Cambio de distribución	Prioridades poco confiables.	Monitor, diagnóstico y reversión con mantenimiento.
Métricas usadas para sancionar	Desconfianza y ocultamiento de incidentes.	Gerencia acuerda propósito, acceso y mecanismos de revisión.
Exposición de API o archivos	Acceso indebido y ejecución de artefactos no confiables.	TI restringe red y carga solo modelos de origen verificado.
Dependencia de una persona	Servicio sin continuidad.	Responsable suplente, versiones y manual de reconstrucción.

No hay datos personales en el dataset. En el piloto real, órdenes de trabajo pueden identificar empleados: minimizar atributos, limitar accesos y separar indicadores de activos de evaluaciones personales. No se incluyen inferencias sobre desempeño individual.

La estrategia define continuidad operativa; Kanban limita capacidad comprometida; ADKAR atiende confianza y práctica; MLOps impide promover candidatos fuera del criterio y vigila cambios. Una buena métrica técnica no basta si no existe personal para inspeccionar las alertas.

No se concluye causalidad: los datos fueron generados con relaciones elegidas por diseño. Un valor alto de una métrica solo acredita que el algoritmo recupera parte de esa señal. No permite afirmar que la antigüedad o el mantenimiento causan fallas ni estimar vida útil.

7. Conclusiones

El prototipo ejecuta validación, experimentación registrada, selección, puerta de calidad, API local, pruebas adversas y monitoreo. La exactitud del modelo base ocultaba baja detección de fallas. La comparación permitió explicitar el intercambio entre recall y carga de inspección.

Las siete etapas se vinculan así: DEFINICION fija la pregunta; datos y esquema evitan entradas corruptas; entrenar y MLflow conservan experimentos; la puerta controla promoción; modelo y Dockerfile describen empaquetado; app expone inferencia; monitoreo y REENTRENAMIENTO vigilan continuidad. Sin cada pieza habría, respectivamente, objetivo ambiguo, datos inválidos, resultados sin trazabilidad, promoción defectuosa, entorno inconsistente, modelo inutilizable o degradación inadvertida.

Docker y CI ya se ejecutaron: contenedor verificado y Actions en verde y en rojo. Falta probar adopción, impacto económico y desempeño con equipos reales. La siguiente decisión es diseñar un piloto en sombra con datos comparables. Si el volumen de falsas alarmas excede la capacidad, revisar el umbral con datos de validación y costos reales.

Declaración de IA: OpenAI Codex apoyó interpretación, diseño, programación, pruebas y redacción. Los resultados técnicos proceden de ejecuciones locales. El equipo debe revisarlos y sustentarlos. No se inventaron entrevistas, aprobaciones, capturas de GitHub ni beneficios empresariales.

Referencias

[1] Holguín García, S. A. Taller integrador de la Unidad 3. Universidad de Caldas. PDF proporcionado por el usuario; sin fecha indicada.

[2] Kanban Guides (2025). The Kanban Guide, edición mayo. https://kanbanguides.org/the-kanban-guide/

[3] Scikit-learn. Common pitfalls and recommended practices. https://scikit-learn.org/stable/common_pitfalls.html

[4] MLflow. Tracking experiments with local database. https://www.mlflow.org/docs/latest/ml/tracking/tutorials/local-database/

[5] Wilson, E. B. (1927). Probable Inference, the Law of Succession, and Statistical Inference. JASA, 22(158), 209-212. https://doi.org/10.1080/01621459.1927.10502953

[6] FastAPI. Testing. https://fastapi.tiangolo.com/tutorial/testing/

[7] GitHub Docs. Building and testing Python. https://docs.github.com/en/actions/tutorials/build-and-test-code/python

[8] SciPy. ks_2samp. https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.ks_2samp.html

[9] Holm, S. (1979). A Simple Sequentially Rejective Multiple Test Procedure. Scandinavian Journal of Statistics, 6(2), 65-70. https://sci2s.ugr.es/keel/pdf/specific/articulo/0052_001.pdf

[10] Prosci. The Prosci ADKAR Model. https://www.prosci.com/methodology/adkar

[11] Material del curso, Unidad 3.3. Gestión del cambio. https://serhol17.github.io/Innovaci-nUcaldas/unidad3-3/

Documentación web consultada en octubre de 2026. Las decisiones de diseño se identifican como propuestas. Fuentes de evidencia técnica: archivos de evidencias y commits del proyecto local.

Anexo A. Comparación y prueba reservada

Figura A1. Precisión y recall en validación. Datos de ejecuciones reales en MLflow; cinco variantes, mismo conjunto de validación.

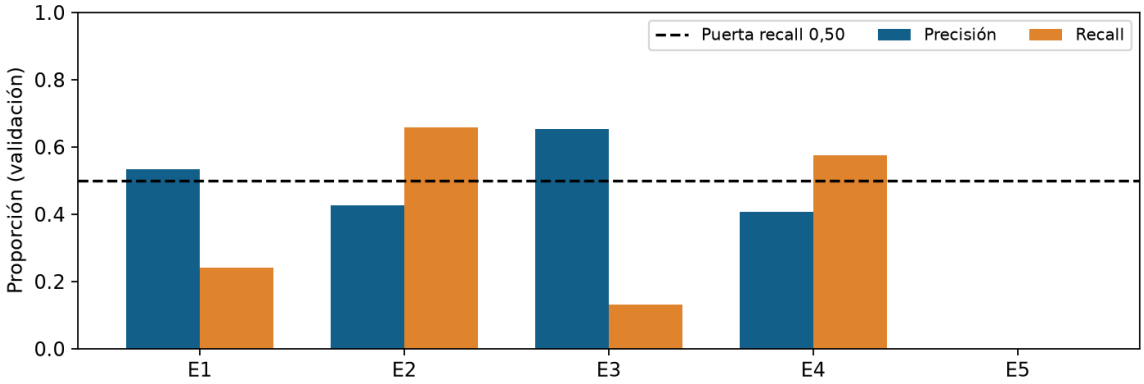
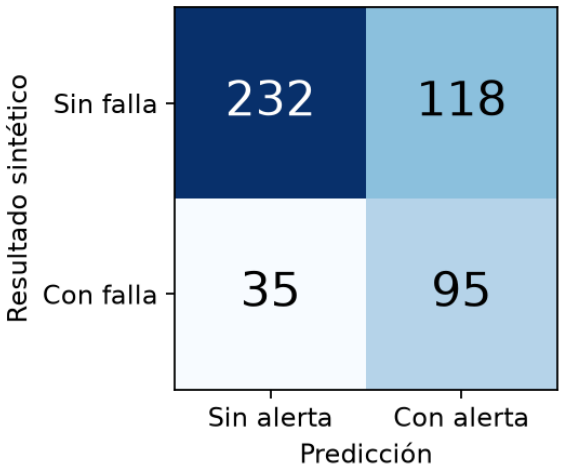


Figura A2. Matriz de confusión del campeón en 480 observaciones reservadas. Datos sintéticos, evaluación local.



La línea de la figura A1 es la puerta de recall, no una meta de precisión. Estas figuras provienen de los registros de ejecución, no de una simulación de interfaz.

Anexo B. Trazabilidad de ejecución

Tabla B1. Evidencia verificable y estado

Pieza	Resultado y ubicación
Datos	datos.json: 2.400 filas, 647 positivos, cero vacíos y hash SHA-256.
Experimentos	experimentos.csv y MLflow SQLite. Cinco variantes y rechazo forzado adicional.
Puerta	puerta.json: rechazo con código 1 y campeón inalterado.
Pruebas	15 pasan; pruebas_verdes.xml. Mutación detectada en pruebas_rojas_mutacion.log.
API local	http.json: salud 200, predicción 200 y datos inválidos 422.
Deriva	monitoreo.json: solo temperatura. Lote control sin alertas.
Git y agilidad	Commits locales; kanban_eventos.jsonl y agilidad.json.
Docker y GitHub Actions	Ejecutados: imagen construida y API verificada; Actions en verde y en rojo. Pegar capturas en el anexo.

Hash del dataset:
cfdfb2b08a0f8a3147a8017f823e5421db0bc75909f51b1c8c90bea915939291

Hash del campeón:
105930208d7d948f298f7c32860a3b256b36b2c786808129dca316772f38d62e

Antes de entregar: incorporar captura de MLflow, API y pruebas, contenedor activo y ejecuciones verde/roja en GitHub. Mantener máximo cuatro páginas de anexos. Estas tablas identifican evidencia local y no sustituyen las capturas pendientes exigidas por el docente.